Lab 2 — Linux Commands Guide

1. File and Directory Commands

`touch` — Create an Empty File

`touch` creates an empty file if it doesn't exist, or updates the timestamp of an existing file.

```
touch ceng111.txt          # create an empty file
touch ceng111.txt          # run again -- content unchanged, timestamps updated
ls -l  ceng111.txt         # shows the modification timestamp
ls -lu ceng111.txt         # shows the access timestamp
touch a.txt b.txt c.txt    # create several files at once
```

`mkdir` / `rmdir` — Create and Remove Directories

```
mkdir cengstuff            # create a single directory
rmdir empty_folder         # remove a directory -- only works if empty
```

`rmdir` refuses to delete a directory that still contains files or subdirectories. Use `rm -r` for non-empty directories (see below).

`rm` — Remove Files or Directories

```
rm file.txt          # delete a single file
rm cengstuff         # error: rm cannot remove a directory without -r
rm -r cengstuff      # recursively delete a directory and all its contents
rm -rf ~/Downloads/*     # force-delete all under Downloads
```

`cp` — Copy Files or Directories

cp copies files and directories. Use `-r` to copy directories.

```
cp dosya ~/Desktop/         # copy a file into Desktop
cp -r ~/Documents/ceng111/lab2 ~/Desktop/    # copy an entire directory
cp file.txt backup.txt      # copy with a different name (same directory)
```

Tip — trailing slash: writing `cp dosya ~/Desktop/` (with the last `/`) tells `cp` the destination must be a directory. Without the slash, if `Desktop` did not exist, `cp` would silently create a file *named* `Desktop` instead of copying into it.

`mv` — Move

Moving and renaming are the same operation on Linux. Unlike `cp`, `mv` does **not** need `-r` for directories.

```
mv Projects/Lily /home/lily/     # move a directory to another location
mv dosya DoSyA    # rename (Linux is case-sensitive: dosya != DoSyA)
mv ceng111 ceng111.bak  # rename a directory; "." in names is perfectly valid
```

2. Listing Files in Detail

`ls` — Options and Arguments

Commands can take both **options** (flags starting with `-`) and **arguments**. In `ls -l ~/Desktop`, `-l` is the option and `~/Desktop` is the argument.

```
ls -a           # show hidden files (names starting with .)
ls -l           # detailed listing: permissions, owner, size, date
ls -al          # combine -a and -l
ls Desktop      # list the Desktop directory (argument)
ls ~            # list home directory
ls ~/Desktop
ls ..           # relative path: list the parent directory
ls /usr/local/  # absolute path
ls .            # '.' refers to the current directory
ls -al ~/.bashrc   # argument is a specific file -- shows only that file
```

Reading `ls -l` Output

```
drwxr-xr-x  6  ali  veli  4096  2009-10-04 12:23  tmp
```

- **First character** — `d` means directory; `-` means regular file.
- **Next 9 characters** — permission bits in three groups of three (owner / group / others). See Section 3.
- **6** — hard link count
- **ali** — owner; **veli** — group.
- **4096** — size in bytes. For directories this number is *not* the total size of the contents; use `du` for that.
- **2009-10-04 12:23** — last modification time.
- **tmp** — file or directory name.

3. File Permissions and `chmod`

Every file has three permission bits — **r**ead, **w**rite, **e**xecute — for three categories of users:

Symbol	Octal value	Meaning
`r`	4	read the file / list the directory
`w`	2	write to the file / create or delete inside the directory
`x`	1	run the file as a program / enter the directory

The nine bits shown by `ls -l` are split as `[owner][group][others]`. For example `rwxr-x--` means: owner can read/write/execute; group can read/execute; others have no access.

Octal notation

Add the values for each category. `chmod 754 dosya` sets owner=7 (rwx), group=5 (r-x), others=4 (r–).

```
chmod 755 script.sh    # owner: rwx, group: r-x, others: r-x
chmod 700 secret/       # only the owner can access this directory
chmod 000 dosya        # no permissions for anyone
chmod 100 cengstuff    # execute only: cd into it but cannot list or write
chmod 300 cengstuff    # write+execute: can create files but cannot ls
chmod 700 cengstuff    # owner has full access
```

Symbolic notation

```
chmod +x script.sh      # add execute for all (u, g, o)
chmod u+x script.sh     # add execute for owner only
chmod g-w dosya         # remove write from group
chmod o-r dosya         # remove read from others
chmod u+w,g-w,o-r dosya   # multiple changes at once
chmod a+x proje.sh      # 'a' = all (same as ugo)
chmod -R 700 my_directory    # Recursively set permissions on directory
```

Escape Character

The backslash (\) is the **escape character** in Linux. It cancels the special meaning of the character immediately following it, treating it as a literal character.

```
cd "ceng 111"    # option 1: wrap in quotes
cd ceng\ 111     # option 2: escape the space with backslash
cd ceng\ \ 111   # two spaces
```

4. Viewing File Contents

`cat` — Print Entire File

`cat` dumps the complete contents of a file to the terminal. It only works on regular files, not directories.

```
cat index.html       # print file contents
cat Desktop          # error: Desktop is a directory
cat file1 file2 > combined.txt    # Combine multiple files into one
```

`more` / `less` — Paginated Viewing

Both let you scroll through a file that is too long to fit on one screen.

```
more long_file.txt   # view long file one page at a time
less long_file.txt   # scroll freely with arrow keys / Page Up / Page Down
```

`echo` — Print Text or Variable Values

`echo` displays text or writes it to files. It's also used to display variables.

```
echo "Hello world"       # print a string
echo $USER               # print the current username (variable)
echo $(date)             # print the output of the date command
echo "This is important" >> notes.txt    # Append text to an existing file
echo $PATH               # Display value of PATH environment variable
```

`grep` — Search Inside Files

The `grep` command finds specific text within files. It supports recursive searches,case-insensitive searches, and more.

```
grep "catnip" luna_notes.txt # Search for "catnip" in luna_notes.txt
grep -i "lucy" *        # Case-insensitive search across files
grep -r "lily" /home    # Recursive search in all files under /home
grep -l "project" *.txt # Show only names of files containing "project"
```

5. Disk Usage and Manual Pages

`du` — Disk Usage

The `du` command shows the disk usage of files and directories. The `-h` option makes output human-readable.

```
du -sh .          # total disk usage of the current directory
du -sh file.txt   # disk usage of a single file
du -sh *          # usage of every file and directory in current location
du -h --max-depth=1 /home/luna    # Show size of top-level items in /home/luna
```

`man` — Manual Pages

`man` provides detailed documentation for commands, including options and examples.

```
man ls      # manual page for ls
man chmod   # manual page for chmod
man grep    # manual page for grep
man -k network   # Search manual pages for commands related to "network"
```

6. Standard Input, Output, and Error

Every running program has three standard streams:

Stream	Number	Default destination
stdin (standard input)	0	keyboard
stdout (standard output)	1	terminal screen
stderr (standard error)	2	terminal screen

These streams can be **redirected** to or from files.

```
ls > a.txt          # redirect stdout to a file (overwrites)
ls >> a.txt         # append stdout to a file
cat < ceng.html     # take stdin from a file instead of the keyboard
cat < index.html > yeni_index.html   # redirect both input and output
ls non_existent_dir 2> hata.txt   # capture error message in a file
history 20 > my_history.txt
```

Pipe `|`

The pipe character `|` sends the **stdout** of one command directly to the **stdin** of the next. This lets you chain commands without writing intermediate files.

```
ps aux | grep firefox          # list only lines containing "firefox"
cat index.html | grep METU    # search an HTML file for "METU"
ls > a.txt                    # redirect to file  (> goes to a file)
ls | grep txt                 # pipe to command   (| goes to a command)
```

7. Process Management

`ps aux` — View Running Processes

The `ps aux` command lists all active processes with details such as PID, CPU, and memory usage.

```
ps aux         # List all active processes
ps aux | grep firefox          # Filter results for processes with "firefox"
```

`kill` — Terminate a Process

The `kill` command stops a process by its Process ID (PID).

```
kill 1234 # Terminate process with PID 1234
kill -9 5678 # Force terminate process with PID 5678
```